Autonomous Agents and Multiagent Systems

A series of notes on the course AAaMS as taught in Politecnico di Milano by Francesco Amigoni and someone else

Autonomous Agents and Multiagent Systems
Negotiation Part 1 - 08/10
- Negotiation
- Cooperative/Axiomatic Approach
  - Properties we want the Bargain Solution to satisfy

Negotiation Part 2 - 11/10
- Rubinstein's Alternating Offer Protocol
- Monotonic Concession Protocol
- Zeuthen Strategy

Computational Social Choice - 18/10
- Social Welfare Function
  - Arrow's Theorem
- Social Choice Function
  - Voting Rule

Computational Social Choice - 22/10
  - Scoring Rules
  - Pile Wise Majority Voting
  - Copeland's Rule
  - Plurality with Runoff
  - Single Transferable Vote (STV)
  - Sequential Majority Voting
  - Strategic Manipulation of vote
  - Probabilistic Voting Rule

Auctions - 25/10
  - Canonical Auctions
  - Common value & IPV
  - Important Considerations
  - Auctioneer's Point of View

Multiple Items Auctions - 29/10
- Introduction
- Winner Determination Problem

Lecture 8/11
Computational Coalition Formulation Part 1 - 12/11
- Characteristic Function & TU Games
  - Superadditivity
  - Convex Property
  - Simple Property
- How to choose the payoff vector?
  - Core
  - Shapley Value

CCF Part 2 - 15/11
- Best Coalition Structure
  - Centralized Algorithm based on DP
  - Distributed Algorithm (Shehory - Kraus)

Multi Agent Planning - 19/11

Single agent planning
Multi agent planning
Families of approaches to Multi Agent Planning
Stochasticity in Multi Agent Planning
DecPOMDPs
Shapley Values Exercise Session - 22/11
Multi Agent Planning Part 2 - 26/11

Negotiation Part 1 - 08/10

Negotiation

Each agent has interests that conflict with the interests of other agents (conflicting interests).

- Agents
  we assume we have only 2 agents but what we say can be generalized to n agents.
  Let's call the agents a and b.
- Issues
  the variables over which the negotiation takes place.
  There can be several issues (price, insurance, modalities in which you pay for example for buying a car).
  We will treat negotiations over single issues.
- Outcomes
  Results of your negotiation process.

$$\Theta$$

We need to define the conflicting measures:

- Utility Function
  given the set of outcomes (the domain is the set of outcomes) and as codomains the set of real numbers.
  given an outcome the value of that outcome is different for agent a and agent b.

$$U^i : \Theta \to \mathbb{R}$$

- Protocol
  set of rules of the game.
  The way the negotiation must be conducted cannot be violated in general (example: if you are the seller, you offer should decrease, if you are the buyers, your offer should increase).
- Strategies
  policies the single agents will adopt for conducting the negotiation.
  within the protocol, the agent decides what to do (example: the seller decides yo offer 5 less every time, the buyer to offer 10 more every time).
  the strategies are decided by the agents.

Cooperative Approach (Axiomatic Approach)

The negotiation does not actually take place, you know everything about the agents, you impose some properties over the final result you would like to reach.
You can calculate a priori what is the final result. If a knows a set of properties , then I can come up with a solution analytically.
The properties that you are imposing on the solution are like axioms.

We will focus on the approach just described.

Non Cooperative Approach (Algorithmic Approach)

You run the negotiation process, you exchange offers and counteroffers until you reach some agreement. Every agent can actually do what they want, there is no agreement over the agents.

Cooperative/Axiomatic Approach

This approach has this structure:

1. We list a number of properties/characteristics that our solution must have
2. We come up with a solution that respects these properties

We assume that the 2 agents are perfectly rational: they prefer outcomes according to their utility function.

We assume that the set of outcomes is partitioned in:

- A : set of possible ***agreements*** (all the legal prices for example).
- D : the ***disagreement***, it is a single element. It is like a special outcome that says that no agreement was found (a special price that says that no agreement was found).

$$\Theta = A \cup D$$

- $S =$ ***Bargaining Set***: set of utilities $S = \{(U^a(z), U^b(z)) | z \in A\}$
- d : set of utilities of the agents when the disagreement will happen.

$$d = (d^a, d^b) = (U^a(D), U^b(D))$$

Bargaining Problem

A Bargaining problem is define as the pair (S, d).

the solution to such problem is defined as the result of a function f defined as follows:

given a Bargaining problem f returns *one* element in S.

$$\text{Bargaining Solution} = f : (S, d) \rightarrow (U^a(z), U^b(z)) \in S$$

$Domain$; $Codomain$.

A negotiation can be viewed as a search in the space of the pairs of utilities connected to the agreement.
f just picks up one possibility in the bargaining set. the output of f is the ***bargaining solution***.

Assumptions on the Bargaining Set S

We assume that S is *closed* and *convex*.

- <u>Convex</u>
  if we take 2 elements belonging to S, their linear combination is still in S.

$$x = (x^a, x^b); \quad y = (y^a, y^b) \qquad x, y \in S$$
$$\theta x + (1 - \theta) y \in S \qquad \theta \in [0, 1]$$

We also assume that there is always an agreement for both agents that will be at least as good as the disagreement $\rightarrow$ players will always prefer the agreements with respect to the disagreements $\rightarrow$ it is not possible that the disagreement is the best possible result.

$$S \cap \{(x^a, x^b) | x^a \geq d^a \wedge x^b \geq d^b\} \neq \emptyset$$

(Assuming this we can avoid assuming that S contains the disagreement).

Properties we want the Bargain Solution to satisfy

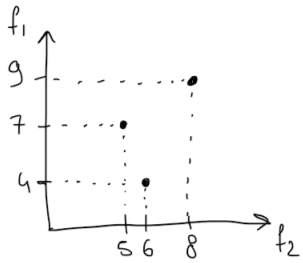
Now we can start to discuss the theory that will bring us to the final result of axiomatic approach.
We will list a number of properties we would like the bargains solution to satisfy.
At the end we will discover that if we want our bargain solution to satisfy all these properties then the bargains solution will have a certain form.
First we need to introduce Pareto Efficiency, then I'll list all the properties.

Pareto Efficiency

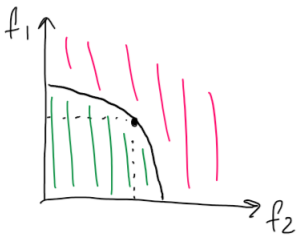
if you are maximizing a single objective function 5 and 7, you choose 7.
Now assume you are maximizing 2 objective functions:

$(5,7),(8,9),(6,4)$ if we have to compare the first and the third, which one is best?
You cannot compare them:
If we consider the first pair and the third pair we notice that 6 is better than 5 but 4 is not better than 7.
Surely I can say that the second pair of utilities is better than the first and the third one.
So we say that $(8,9)$ dominates both first and third pairs.
Moreover, we notice 2 possibilities:

- One dominates the others
- No comparison is possible:
  You cannot have a total ordering over the elements because some elements cannot be ordered.



(5,7)
(8,9)
(6,4)



on the frontier → you cannot compare
inside → dominated
outside → dominates

The bargaining solutions is such that:

$$f(S,d) : \nexists (y^a, y^b) \in S | (y^a, y^b) \geq (x^a, x^b) \text{ and } y^i > x^i$$

Considering a specific agent i.

Meaning: *It makes no sense to select an outcome that is dominated by another one*.

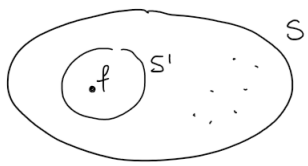
Properties

1. <u>Individual Rationality</u>

$$f(S,d) \geq d$$

Bargaining Solution; Bargaining Problem.
I will accept nothing that gives me less than the disagreement.

2. Symmetry
   The outcome should not change if the utilities are the same but I swap the names of the agents.
3. Efficiency
   The bargaining solution returned by the function f must be *Pareto Efficient*.
   The meaning of Pareto Efficiency is explained right after the listing of these properties.
4. Invariance
   The solution that is returned by f should not change if the utilities are linear changes.
   if I multiply all the utilities by α, then the final outcome that is selected as the bargaining solution will be multiplied by $\alpha \rightarrow$ it scales but does not change.
5. Independence of irrelevant alternatives
   $\forall S' \subseteq S \quad f(S,d) \in S' \qquad f(S,d) = f(S',D)$



   My result does not depend on what I left out.

The final result is the Nash Bargaining Solution:
If you would like to find a bargaining solution f that satisfies all these 5 properties you have only 1 possibility: Your bargaining solution must be:

$$f(s,D) = \arg\max(x^a, d^a)(x^b - d^b)$$

$$x = (x^a, x^b) \text{ with } x \in S$$

$$d = (d^a, d^b) \text{ with } x \geq d$$

Such solution is unique.
In this case you don't have to negotiate.
If you are in a set in which you know everything and you want a solution that satisfies all these properties, you don't need to run a negotiation: just compute the solution.
This is what we mean with Axiomatic $\rightarrow$ no need to run the iterative process, just apply the formula!

PARETO dominates

3,3	0,5
5,0	1,1

NASH EQUILIBRIUM
is this outcome Pareto dominated by another outcome? Yes by 3,3

any other pareto optimal outcomes? no

Negotiation Part 2 - 11/10

We will now focus on the agreement found by the agent, not on the policy.

We'll define a protocol, see how the agents behave in it, and the study the possible outcome that will be reached by the agent.

Let's start with the first protocol:

Rubinstein's Alternating Offer Protocol

I make an offer, the other agent makes a counter offer, I make a new offer, and so on, until we find some agreement.

We assume that we have 2 agents, a, b.

We assume there is one issue over which the agents are negotiating. It can be though as an object that can be divided by the 2 agents.
Let's think of a cake. It can be divided by the 2 agents. We'll call the piece of cake that goes to the agent a x^a. x^b is the part of the cake that goes to b.

we assume that $x^a + x^b = 1$ (it forms the complete cake).

The set of outcomes is trivially the set of all the pairs (x^a, x^b) such that $x^a + x^b = 1$.

in math: $O = (x^a, x^b) | x^a + x^b = 1$ where O stands the set of possible Outcomes.

We assume that time is evolving in discrete period: $t = 1, 2, \ldots$.

We assume that players play in turn, with player a playing first:
So agents a makes offers in odd timestamps, while b makes offers in even timestamps. This is the core of "Alternating" Offer protocol.

the ones above are all and only the assumptions that the agents must do.

Let's talk about the utility of a: it depends on how big the cake is and on the timestamp in which the offer is agreed.

$$U^a(x^a, t) = x^a \cdot \delta_a^{t-1}$$

where δ_a is the discount factor and $0 \leq \delta_a \leq 1$

$\delta = 1 \rightarrow$ the agent is very patient.

$\delta = 0 \rightarrow$ the agent must accept the first offer otherwise it gets zero.

and obviously:

$$U^b(x^b, t) = x^b \cdot \delta_b^{t-1}$$

Rubinstein shows that it is possible to compute a subgame perfect equilibrium:

$$x^a = \frac{1 - \delta_b}{1 - \delta_a \delta_b}$$

$$x^b = \frac{\delta_b}{1 - \delta_a \delta_b}$$

It means that the agent will not find convenient to deviate from these values.

we notice that if

$$\delta_a = 0; \delta_b = 1 \rightarrow x_a = 0;\ x_b = 1$$

$$\delta_a = 1; \delta_b = 0 \rightarrow x_a = 1;\ x_b = 0$$

$$\delta_a = 0.5; \delta_b = 0.5 \rightarrow x_a = \frac{2}{3};\ x_b = \frac{1}{3}$$

in the last case, the agent a gets more cake because it started first.

The cases above have been computed considering that the negotiation can go on forever, but what happens if we have a deadline $t = n$? (n is the last timestamp in which an offer can be done).

Let's suppose even that $\delta_a = \delta_b = \delta$

$n = 1 \rightarrow$ agent b will just accept, can't do anything else. So agent a will propose $offer(1, 0)$ more precisely:

$$\text{STRAT-A}(1) = \text{OFFER}(1, 0)$$
$$\text{STRAT-B}(1) = ACCEPT$$

$n = 2$

$$\text{STRAT-A}(1) = \text{OFFER}(1 - \delta, \delta)$$
$$\text{STRAT-A}(1) = \text{ACCEPT}$$

which translates into:

$$U^a(1 - \delta, 1) = 1 - \delta$$
$$U^b(\delta, 1) = \delta$$

let's revise what we have just written:
The agent a prefers to offer to b δ part of the cake because the utility of b can't be more than δ (in fact $U^b(1, 2) = \delta$), because this way a gets at least $1 - \delta$ as utility, so it's a win win.

let's generalize:

deadline n, $\delta_a = \delta_b = \delta$

$$\text{STRAT-A}(n) = \begin{cases} \text{if a's turn: OFFER(1,0)} \\ \text{if b's turn: ACCEPT} \end{cases}$$

$$U^a(n) = \begin{cases} \delta^{n-1} & \text{if it's a's turn} \\ 0 & \text{if it's b's turn} \end{cases}$$

$t < n$

$$\text{STRAT-A}(t) = \begin{cases} \text{if a's turn: OFFER}(1 - \delta x^b(t+1), \delta x^b(t+1)) \\ \text{if b's turn: if } U^a(x^a, t) \geq UA(t+1) \text{ then ACCEPT else REJECT} \end{cases}$$

Monotonic Concession Protocol

- agents a, b ; U^a, U^b
- O
- simultaneous offer protocol
- $t = 1, 2, \ldots$

From the point of view of agent a:

1. $x^{(a)} \leftarrow \arg\max_{x \in O} U^a(x)$

2. propose $x^{(a)}$

3. receive $x^{(b)}$

4. if $U^a(x^{(b)} \geq U^a(x^{(a)}))$ then ACCEPT $x^{(b)}$ and RETURN
else $x^{(a)} \leftarrow x \in O$ such that $U^b(x) \geq U^b(x^{(a)})$ and $U^{(a)} \geq 0$

5. goto 2

This algorithm can be applied only when you know the utility of the other agent, which does not happen very often in the real world...

Moreover, the converge of this algorithm can b very long.

Another problem: It can happen that agent a accepts the offer of agent b and agent a accepts the offer of agent b, but they are not the same! we have two different agreements: we need to introduce some tie-breaking to decide the agreement. we can pickup just one at random , or we can find the middle point of the offer and come up with a single agreement.

There is a variant of this protocol that is called the Zeuthen Strategy

Zeuthen Strategy

This is the point of view of a

1. $x^{(a)} \leftarrow \arg\max_{x \in O} U^a(x)$

2. propose $x^{(a)}$

3. receive $x^{(b)}$

4. if $U^a(x^{(b)} \geq U^a(x^{(a)}))$ then ACCEPT $x^{(b)}$ and RETURN

5. $\text{risk}_a \leftarrow \frac{U^a(x^{(a)}) - U^a(x^{(b)})}{U^a(x^{(a)})}; \text{risk}_b \leftarrow \frac{U^b(x^{(b)}) - U^b(x^{(a)})}{U^b(x^{(b)})}$

6. if $\text{risk}_a < \text{risk}_b$ then $x^{(a)} \leftarrow x \in O$ such that $\text{risk}_a > \text{risk}_b$

7. goto 2

Property: the final agreement found via this algorithm is a pareto optimal agreement, which means that there is no another agreement that is not worse for one agent and strictly better for the other one.

it finds an agreements which is a Nash BARGAININ solution. It is an agreement that maximizes the product of the two utilities.

Computational Social Choice - 18/10

Applications:

- Aggregation of preferences.
- Formation of coalitions.
- Sometimes you have multiple robots that have to keep a formation, for example military patterns.
  It is based on the fact that there is a leader that is moving and all the other robot are keeping a fixed distance and angle from him. They should also avoid to collide with each

other.
What happens if the leader fails? In this case there is a kind of voting for choosing the new leader, and this is the social choice.

Let's start from the model that we will adopt for studying computational social choice.

we'll have:

- a number of ***agents*** $N = \{1, 2..., n\}$
- a set of **alternatives**, also called **candidates** (in the case of political elections for example) U. we assume $|U| \geq 2$.
- ***preferences***, denoted by the symbol *preference relation* $>_{\sim i}$ (it is an ordering over the alternatives).
  for it being a proper ordering we need to constraint such relations in the following way. we need to say that such relation is
  - complete: $\forall a, b \in U : a >_{\sim i} b \text{ or } b >_{\sim i} a$ (the or is not generally exclusive)
  - transitive: $\forall a, b, c \in U : \text{if } a >_{\sim i} b \text{ and } b >_{\sim i} c \text{ then } a >_{\sim i} c$

Observations: agents and candidates could be disjoint or not depending on the case. In political election agents are people and politicians, while candidates are politicians.

Now let's introduce some symbols:

- ***preference profile*** $R = (>_{\sim 1}, >_{\sim 2}, >_{\sim n})$ (it is an array of preferences, one for each agent). a preference can be structured as a pair of agent, candidate. So a preference profile can be represented as a list of pairs.
- a ***set*** of all possible ***preference relations over*** $U : \mathscr{R}(U)$
  $>_{\sim i} \in \mathscr{R}(U)$

Mathematicians game:
We are at a party in which people can drink only one beverage.
6 mathematicians want to drink, in preference order, milk,wine,beer.
5 mathematicians want to drink, in preference order, beer,wine,milk.
4 mathematicians want to drink, in preference order, wine, beer,milk.
So:

- we have 15 agents: $n = 15$.
- we have 3 alternatives: $|U| = 3$
- for the mathematician number 1, who belongs to the 6 mathematicians cited first, we'll have $\text{milk} >_{\sim 1} \text{wine} >_{\sim 1} \text{beer}$. and so on for all the mathematicians.

It is not easy to aggregate together different preference relations with a global choice that is satisfactory for all people. It depends even on the mechanism of voting.

Social Welfare Function

It is a function that given a preference profile returns a preference relation.

$f : \mathscr{R}(U)^n \rightarrow \mathscr{R}(U)$

$f(R) \ = \ >_{\sim}$

This is what the function should do, there are different ways to implement it.

We want this function to have the following properties

- <u>Pareto Optimality</u>
  Informal definition: assume that every agent thinks that candidate a is better than b. then it is reasonable to assume that a will be better than b even in the global relation.
  Formal definition:
  $\text{if } a >_i b \text{ then } a > b$
  the one above has been written assuming the following: $a >_{\sim i} b \text{ but not } b >_{\sim i} a$, which is not the general case.
  the consequence assumes instead that: $a >_\sim b \text{ but not } b >_\sim a$.
  I had to write the formal definition this way because the actual formal definition would be complex to be written, so Amigoni preferred specifying the assumptions afterwards.
- <u>Independence of irrelevant alternatives (IIA)</u>
  imagine this starting situation

$$f \begin{cases} \ldots >_{\sim 1} a >_{\sim 1} \ldots >_{\sim 1} b >_{\sim 1} \ldots \\ \ldots >_{\sim 2} a >_{\sim 2} \ldots >_{\sim 2} b >_{\sim 2} \ldots \\ \ldots >_{\sim 3} a >_{\sim 3} \ldots >_{\sim 3} b >_{\sim 3} \ldots \end{cases} \rightarrow \ldots >_\sim a >_\sim \ldots >_\sim b >_\sim \ldots$$

  In a nutshell we want to say that if you swap the red and yellow elements, and, as we can see, a and b have the same ordering over all the agents, and in case a and b have anyways the same order in the global relation, we can say that our formulation has IIA.

  In case of IIA the ordering of a and b in the global relation does not dependent on how it is called the yellow element.
  Formally:
  $\text{if } R|\{a,b\} = R'|\{a,b\} \text{ then } >_\sim |\{a,b\} =>'_\sim |\{a,b\} \text{ where } f(R) =>_\sim \text{ and } f(R') =>'_\sim$
- <u>Non-Dictatorship</u>
  A good social welfare function is such that there is no such an agent which imposes the ordering over the other agents. This is Non-Dictatorship.
  Formally:
  $\text{there is no } i \in N \text{ such that } f(R) =>_{\sim i} \forall >_{\sim i}$

Arrow's Theorem

There is no any social welfare function that satisfies the 3 properties above when you have at least 3 candidates.

The property that is mostly dropped is the IIA.

Social Choice Function

The Social Welfare Function can be too powerful in many situations. take for example the problem of electing 3 new members in parliament over 10 possible candidates. you just need to know the 3 most voted ones, you don't care about the others.

So now we'll define the social choice function

$f : \mathscr{R}(U)^n \times \mathscr{F}(U) \rightarrow \mathscr{F}(U)$

where $\mathscr{F}$ is the powerset of U: $\mathscr{F} = 2^U$, which is the set of all feasible sets of alternatives.

$A \in \mathscr{F}(U) \; A \subseteq U$

$f(R, A) = A' \qquad A' \subseteq A$

Voting Rule

one special case of the social choice function is the ***voting rule***.

Given a preference profile it returns a set of candidates.

$f : \mathscr{R}(U)^n \to \mathscr{F}(U)$
$f(R) \to A'$

f is resolute when $|f(R)| = 1$

Computational Social Choice - 22/10

Example

6	5	4
milk	beer	wine
wine	wine	beer
beer	milk	milk

scoring rules: rule that gives some point to each alternative according to the ranking related to each agent.

$$>_\sim (s_1, s_2, \ldots, s_m) \qquad m = |U| \qquad s_i \in \mathbb{R}$$

where s_i is the score of the first alternative in the ranking.

$$s_1 \geq s_2 \geq \ldots \geq s_m \qquad s_1 > s_m$$

Scoring Rules

- Plurality Voting
  Everyone votes for his preferred alternative. for example, milk gets 6 votes, beer 5 votes, and wine 4.
  We are considering one point for each vote.
  $(1, 0, 0, \ldots, 0)$ you consider the first ranked $\to$ milk
- Borda Count
  $(m-1, m-2, \ldots, 0)$
  $m =$ number of alternatives. In our example we have:
  milk: $6 \cdot 2 + 5 \cdot 0 + 4 \cdot 0 = 12$
  wine: $6 \cdot 1 + 5 \cdot 1 + 4 \cdot 2 = 19$
  beer: $6 \cdot 0 + 5 \cdot 2 + 4 \cdot 1 = 14$. shortly we have that the last position counts 0, the second one counts 1, and the first one counts 2.

Which one between Borda and Plurality can be considered also social welfare function?
Which means: which one will return a ranking?
Borda returns a global ranking among preferences.

Pile Wise Majority Voting

A ***Condorcet Winner*** is an alternatives that wins against every other alternative in a pile wise majority voting.

Pile wise majority voting: a voting where you have only 2 alternatives and the alternatives that get more points is the winner.

A Condorcet winner is a very strong candidate because it wins against a every other candidate.

Do we have a Condorcet winner in the beverage example? (we are considering Borda count)

$milk - beer$

$milk - wine$

$wine - beer$

Yes, wine, because it wins against every other candidate.

A ***Condorcet Extension*** is a voting rule that selects a Condorcet winner if a Condorcet winner exists.

It is not the case of plurality, it may be the case of Borda considering the result but it's actually not:
No scoring rule is guaranteed to select the Condorcet winner if the Condorcet winner exists, no scoring rule is a Condorcet Extension.

Example

6	**3**	**4**	**4**
a	c	b	b
b	a	a	c
c	b	c	a

17 agents in total. 3 candidates.

is there any condorcet winner?

$a - b$ (9 over 8)

$a - c$ (10 over 7)

at this point we know a is a condorcet winner.

$b - c$

Let's see that no scoring rule would have selected a:

these are the generical weights associated with preference ranks: $(1, x, 0)$

$a : 6 \cdot 1 + 3 \cdot x + 4 \cdot 0 = 6 + 7x$
$b : 6 \cdot x + b \cdot 0 + +4 \cdot 1 + 4 \cdot 1 = 8 + 6x$
$c : 0 + 0 + 3 + 4 \cdot x = 3 + 4x$

There is no legal (between 0 and 1) value of x such that $6 + 7x > 8 + 6x$, in fact we obtain that $x > 2$ while we would like an x such that $0 < x < 1$.

Copeland's Rule

In order to determine the winner you give to each alternative one point every time it is winning against another alternative, you give $1/2$ when there is a tie, 0 points when it loses.
$a : 2$
$b : 1$
$c : 0$

Plurality with Runoff

So far we considered voting rules that have a single step.
There are multi step voting rules:
In voting rules that evolve in steps, the winner is found after several steps. An example is plurality with runoff:

Plurality with Runoff: typical in political elections.
It is a 2 stage voting.
In the first stage you run a plurality voting.
Then you take the 2 most voted candidates and run a pairwise majority voting between the 2 (a ballot).

Example

6	5	4
milk	beer	wine
wine	wine	beer
beer	milk	milk

after 1st stage:

milk	beer
6	5

after 2nd stage:

beer wins over milk (9 over 6):

Conclusion:

Beer wins.

Single Transferable Vote (STV)

Run a plurality voting, then the alternatives that are ranked first by the lowest number of agents are eliminated, then you run a new plurality voting without considering the eliminated candidates.
N candidates $\rightarrow N-1$ round to perform.

Example

6	5	4
milk	beer	wine
wine	wine	beer
beer	milk	milk

I eliminate wine.

6	5	4
milk	beer	beer

beer **6**	milk **5**	milk **4**

beer wins over milk (9 over 6):

I eliminate milk.

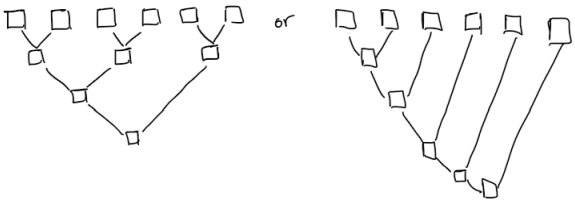
Beer wins.

Is it a condorcet winner extension? no.

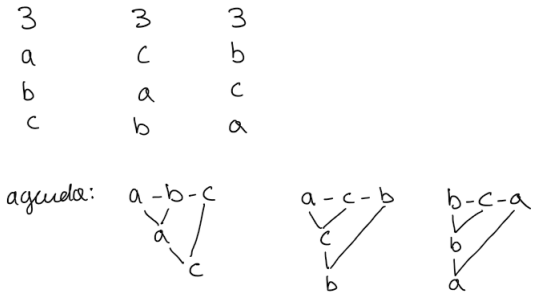
It is not trivial to find a system that selects a good candidate.

Sequential Majority Voting

Combine in different ways a sequence of majority voting.



Agenda : the order in which alternatives are paired in a sequence of pair wise majority voting.



what we are saying in the second sequential majority voting example is:
let's compare a with b first, and then let's compare the winner with the other one. according to different ways to start there are different final outcomes! So whenever someone says "let's start from evaluating those two" there is something wrong with this kind of reasoning.

According to how we select the agenda we have different winners.
Selecting the agenda is not neutral.

Sequential Majority Voting is a condorcet extension because the condorcet winner can never loose in a pair wise.

Strategic Manipulation of vote

Let's consider that we have 4 candidates

1	**2**	**2**	**2**
a	a	b	c
b	c	a	b
c	b	c	d
d	d	a	a

I'm running plurality voting. who is the winner? candidate a because he gets more votes.

Now let's suppose I'm one of the last two players (fourth column).
I'm not happy with the outcomes.
If I know the preference relation of the other candidates what can I do?
I say: if I vote c the winner bill be a. let's vote b, even if it is not my favorite candidate, but it will win, and I like him way more than a. This is called ***strategic manipulation***.

any agent can report wrong information in order to get an advantage

What happens if, in the same example, instead of using plurality voting we use Borda count?

$a : 9$
$b : 14$
$c : 13$
$d : 6$

b wins.

Also Borda count is prone to strategic manipulation..

Gibbard–Satterthwaite Theorem

Any voting system is prone to strategic manipulation.

So you can not avoid strategic manipulation, but you can make it harder to happen.

Probabilistic Voting Rule

So far we have considered deterministic voting systems. The winner is deterministically selected.
It is also possible to use probabilistic voting rules:
The rule is not selecting a candidate, but it is giving in output a probability distribution over the candidates. Then you pick up the winner randomly selecting the candidate according to the probability distribution.
So deterministic voting rule will output something like: the winner is a with 100% probability.
Probabilistic voting rules will output something like: the winner is a with 70%, b with 10% and so on.

Another Gibbard-Satterthwaite Theorem

when

- *there are more than 2 alternatives*
- *you use a probabilistic voting rule*
- *you impose that your probabilistic voting system is Pareto optimal*
- *and it is no manipulable*

then the only probabilistic voting system that is satisfying it is probabilistic dictatorship.

Auctions - 25/10

Auctions ("aste" in italian) are very useful in the real world, for example:

- in the stock market
- public concourse

An auction is made out of an auctioneer (usually the one who sells) and some agents (usually the ones who buy). roles could be the opposite, for example for public concourses, in that case the auctioneer is the one who buys.

When you set up an auction you have to set up the rules of the auction. for example if you are the owner of Ebay you have to say what is the minimum and maximum price or which an user can sell an item.
You have to set even the rules: for example, is it possible to propose a price and get immediately the item? or we must wait for the end of the auction?

What an agent decides to do is called *strategy*.

Different kind of good's auctions

- single-good
  you make an offer only on one item
  (i.e. Van Gogh's painting auction)
- multi-good
  you make an offer on some of a set of goods, or all of them, or just one.
  (i.e. telecommunication auctions)

if you are the buyer you can fix a maximum price that you are willing to offer.
At the same time, if you are the seller, you can fix a minimum price that you are willing to accept.
This last price is called ***reservation price***.

Canonical Auctions

- English Auctions
  The auctioneer announces an initial price (the reservation price).
  The agent can make a new offer with the following constraint: the new offer should be larger than the largest offer made so far by any of the other agents.
  When the auction in this case ends? when there is no new offer.
  The agent will pay exactly the amount of money that he has offered.
  - offer made by agents
  - usually big jump of prices
  - the auctioneer is more passive. he has no control
- Japanese Auctions
  At the beginning all the agents will be standing up. The auctioneer will start from his reservation price and he will call increasing prices.
  If an agent sits down he is out of the auction forever.
  The winner is the last standing agent.
  The amount of money he will pay is the price that the auctioneer has proposed at last.
  What happens if we have two agents standing and they sit down at the same time?
  One tie-breaking mechanism is: a new auction is run just for those agents.
  - offer made by auctioneer.
  - more linear.
  - the auctioneer is more in control of the situation.
- Dutch Auctions
  there is a clock. it does not show the time but the price.
  it starts from a really high price and goes down.
  the auction ends when one of the agents stops the clock.
  The winner is the agent that stopped the clock and the amount of money he will pay is the price that is displayed by the clock at time of stopping.
  (i.e. for selling flowers, for selling fish. it is really fast. used for selling thinks that are not of a big value and you have to sell them fast.
  the auctioneer can set the speed of the clock)

- Sealed-bid Auctions
  the agents are sending their bids (offer) to the auctioneer using a sealed envelope.
  There are several families of sealed-bid auctions, but we'll see just two:
  - ***First Price***
    The auctioneer selects the largest possible offer.
    The agent that made the best offer will get the item and pay exactly the amount he has offered.
  - ***Second Price*** (also called Vickrey auction)
    The auctioneer selects the largest possible offer.
    the winner is the one that made the largest offer but he does not pay his offer, but the second highest.

  (used for allocating public works/concourses). Sealed-Bid auctions are *asynchronous*! no need to have all the agents present at the same time in a specific place.

Common value & IPV

if in an auction I'm selling an 1 euro coin, would you pay more than 1 euro for it? no, if it has no special feature. all the agents agree on a price, that is the ***common value***.
If such coin was instead a very very rare one? it would have a higher value.

Another example: I'm going to buy a house. I've to consider many characteristic depending on my needs (if I'm in a wheelchair I will value it more if it develops on just one floor).
It has a different value depending on the person that is going to buy it.
In the case above we'll then assume that goods have ***independent private values*** (IPV).

Important Considerations

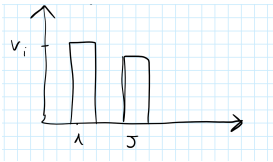
Considerations on Second Price Sealed-Bid Auctions

Now let's do some considerations on Second Price Sealed-Bid Auction:

If I'm using Second Price Sealed-Bid Auction, offering your true evaluation of the object is the dominant strategy (if you think that the object is worth 50, you offer 50).

why? here it is:

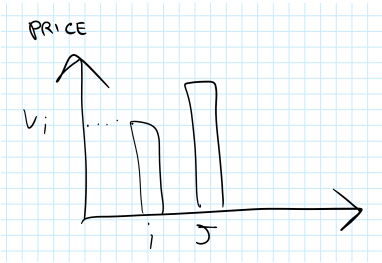
Case 1:
Let's assume that v_i is going to win this auction.
the y axis represents the price paid.



is it wise for i to propose a higher offer? yes, he would still pay the amount offered by j.
Is it wise for i to propose a lower offer? no, he could lose the item.

Case 2:

Let's assume that v_j is going to win this auction.



is it wise for i to propose a higher offer? no because if you offer more than v_i you could end up with gaining the item, but paying more than what you value it (you value it v_i), which is not rational.
is it wise for i to propose a lower offer? no because it would reduce his chances of getting the item.
This is a mechanism that enforces the agents to tell the truth!!

Considerations on English and Japanese Auctions

the winning strategy is to offer an increase until my true evaluation.

Considerations on Dutch and First Price Sealed-Bid Auctions

There is no dominant strategies in this case.

There are though some interesting results:

- strategical equivalence
- equilibrium strategies
- risk neutrality

*Dutch and FPSB are **strategically equivalent**.*

Why?

let's suppose there are 4 agents and I'm involved in a Dutch auction. the price starts at 100 and it reaches 80, which is my true value for the item. should I stop or not?
If I don't know how much the other agents are willing to pay, it could be that it is not the best thing to do to stop the evaluation, because I could end up paying less for the item.
the same happens with FPSB (if I know that the second highest offered value is 70, I can offer 71, even if my true value is 80).

In this cases we can have some ***equilibrium strategies***: if an agent deviates from his strategy and the others are not changing theirs, such agent is going to lose (same thing as Nash Equilibrium).

If you have n agents, the following is an equilibrium strategy:

$$(\frac{n-1}{n}v_1 \frac{n-1}{n}v_2, \dots, \frac{n-1}{n}v_n)$$

where v_i is the true value for agent i (the value he is willing to pay).

Moreover we can say that agents are ***risk neutral***:
Let's explain it via an example.
I have 0 euro in my pockets. you give me a bill of 10 euros. my utility was zero, now it is 10, good. now let's instead assume I have a million euros in my pocket. you give me 10 euros. the increment is exactly the same in both cases, but in the first case I value more passing from 0 to 10, then from 1 million to 1 million and ten. The scenario just described is *not* risk-neutral.
I'm risk neutral if I value in the same way the increment, independent on how much I have.

Auctioneer's Point of View

Let's now talk about the auctioneer:

If I'm going to set up an auction, what mechanism should I choose?

Revenue Equivalence Theorem

n risk-neutral agents.

IPV.

the values v_i are uniformly taken from an interval that is $[\ \underline{v}\ ,\ \overline{v}\ ]$.

if

- the good is allocated to the agent that made the largest evaluation (not the largest offer!)
- the agent with the smallest evaluation has basically no chance to get the good.
  (it's expected utility is set to zero)

then

- any kind of auction that satisfies such preconditions, the provide to the auctioneer the same expected revenue

this means that all the auctions that e have seen satisfy the preconditions, so all of them are equivalent from the point of view of the auctioneer.

things to think about for next lecture

- take the English auction, try to verify that the good is always allocated to the agent with the largest v_i.
- try to think what is the amount of info that agents participating in auction are disclosing to the others in the different auctions we have seen.

this will show me why sealed bit auctions are very popular in real world scenarios.

Multiple Items Auctions - 29/10

Introduction

In this scenario we have that multiple items are auctioned at the same time.

Multiple good/item auctions are combinatorial auctions, they are called this way because we use a combinational approach to solve them.

Example: A company selling stocks in blocks $\rightarrow$ you can make an offer for the first and third stocks block, or for the first and the second one.

the problem is described with the followings:

- **Agents** $N = 1, 2, 3, \ldots, n$
- **Goods** $G = 1, 2, 3, \ldots, m$
- **Valuation function**
  The domain is the power set (set of all possible subsets) of the goods.
  The codomain is a real number.
  To every subset of goods is associated a real number.

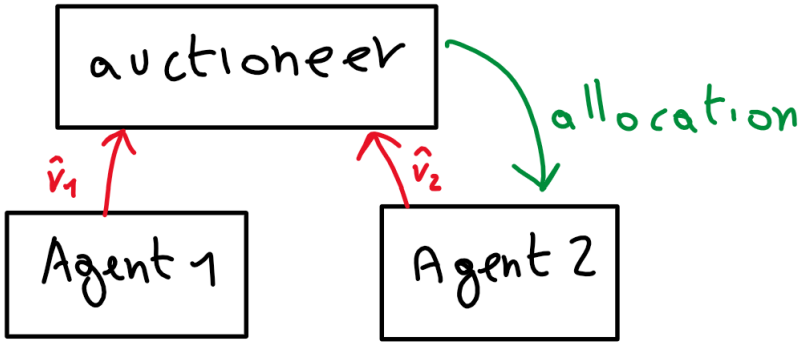
$$v_i : 2^G \rightarrow \mathbb{R}$$

- **Submitted bid of the auctioneer**

$$\hat{v}_i$$

- $v_i - \hat{v}_i$
- Agents can decide not to make an offer for one or a group of goods.
- Agents can offer less than the real value

Extension to the combinatorial auction



Important difference: this time the bid is not a number, it is a function.
The auctioneer will receive all the bids and will decide which agent is the winner $\rightarrow$ difficult to do.

Complementarity

The evaluation function can express some sort of complementarity when the value of the union between two disjoint subsets of G is larger than the sum of the two.
If it holds for every subset it means that agent i thinks that having goods together is better than having it alone.
That means that the two objects are complementary. (Right and left shoes for example).

$$T, S \subseteq G; \quad T \cap S = \{\}; \quad v_i(T \cup S) > v_i(T) + v_i(S)$$

Substitutability

The value that agent i gives to two objects together is less than the value of the sum (two tickets of the same play in different days).

If you are the seller the best thing to do in the substitutability case is to sell one item at the time (same cases of the previous lesson)

Winner Determination Problem

Integer Linear Optimization Problem

Let's make an example showing that complementarity can be a problem.
You are the auctioneer and you receive this offer:

$$N = \{1, 2, \ldots, n\} = \text{set of buyers}$$

$$G = \{1, 2, 3, 4, 5\} = \text{set of items}$$

S = Item	$\bar{v}(S)$ = Best offer for such item/items
1	5
2	4
5	1
1,2	6
1,3,5	7
1,4	5
2,5	10
3,5	2

Some items (or subsets of items) have not received any offer.

Let's introduce some notation:

Considering $S \in G, i \in N$:
The subset of items S goes to agent i if $x_{S,i} = 1$. Otherwise $x_{S,i} = 0$.

X is a vector called allocation, and corresponds to the distribution of $X_{S,i}$.
We have to solve the following integer linear optimization problem:

$$\max \sum_{i \in N} \sum_{S \in G} \hat{v}_i(S) x_{S,i}$$

$x_{S,i} \in \{0, 1\} \quad \forall S \subseteq G \quad \forall i \in N$

Constraint 1: I can sell every item at most to one buyer

$$\sum_{S | j \in S} \sum_{i \in N} x_{S,i} \leq 1 \quad \forall j \in G$$

Constraint 2: To each buyer is associated one and only one subsets of bought items (could be the empty set as well).

$$\sum_{S \in G} x_{S,i} = 1 \quad \forall i \in N$$

As already said this is an *Integer* linear problem. being so, it is more difficult to solve with respect to typical linear optimization problems.
It is proved to be $\mathrm{NP\text{-}Hard}$, so it can be solved optimally only for small instances.
The idea is to find a good solution for the auctioneer, in general.

Algorithm

Step 1: Fill the table

Let's add to the table all the bids we have not received for S of cardinality 1 (single item).

S = Item	$\bar{v}(S)$ = Best offer for such item/items
1	5
2	4
5	1
1,2	6
1,3,5	7
1,4	5
2,5	10
3,5	2
3	0
4	0

Step 2: Tree Building

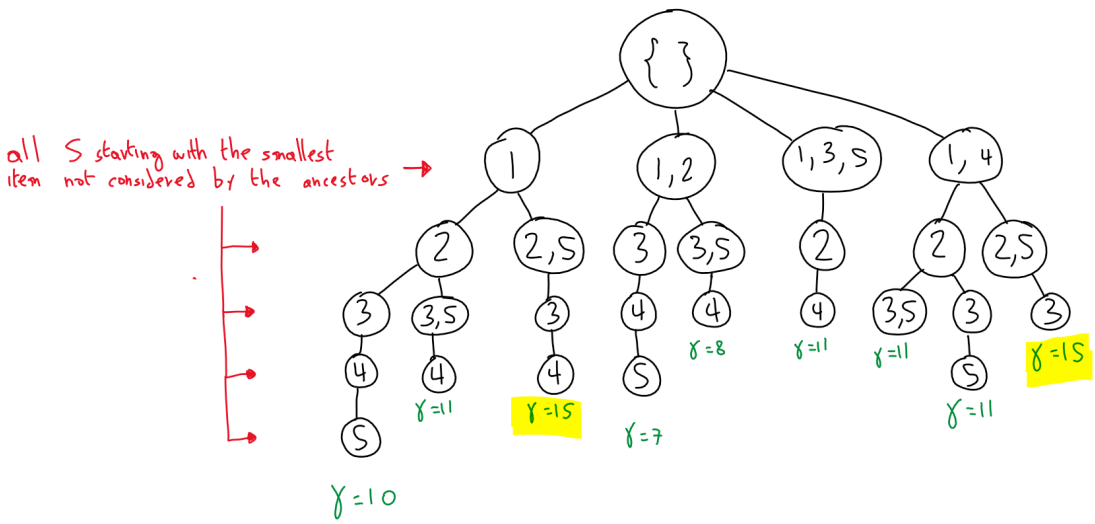
Root = Empty set.
I will add a child for every bid I have received that

1. contains the smallest items (item 1 is considered smaller than item 2, etc) not yet on the path
2. does not contain any item already in the path

$\gamma = \text{cost of a specific path}$

Stop as soon as you have considered all the items in G.

Leaves are all the possible allocations of good (disjoint and containing all the items).



We notice that...to be continued, I'll finish it soon.

Lecture 8/11

To be done, wait for it.

Computational Coalition Formulation Part 1 - 12/11

We are talking about cooperative games:
agents are playing games in which there are two big differences with respect to classical games that are studied in game theory:

- there is some benefit in cooperating
- if agents make some agreement, than this agreement is binding (there are ways outside of the game for enforcing this agreement)

In this kind of games, actions are taken by groups of agents.

We are going to look to a specific kind of cooperative game, called ***Transferable Utility Games***: *the utility is not individual, but given to the group of agents in the coalition* (such coalition can be then divided within the coalition).

We are going to introduce the concept of ***characteristic function games***: *as a group my utility depends only on what I do, not on what other groups do*.

Characteristic Function & TU Games

$$A = \{a_1, a_2, \dots, a_n\} \qquad \text{are agents}$$

$$v : 2^A \to \mathbb{R} \qquad \text{is our characteristic function}$$

such function is called characteristic function and associated a real number to a specific coalition.

assumptions:

$$v(\{\}) = 0$$
$$v(C) \geq 0 \qquad \forall C \subseteq A$$

each C is called coalition.

The whole set of agents A is called sometimes *grand coalition*.

Introductory Example

There are 2 children.
Charlie has 6 dollars.
Marcy has 4 dollars.
Patty has 3 dollars.
Charlie wants to buy ice cream which is 500g and it costs 7 dollars. The others are explained mathematically.
$w_c = 500g;\ c_c = 7\$$
$w_m = 750g;\ c_m = 9\$$
$w_p = 1000g;\ c_p = 11\$$

$A = \{c, m, p\}$
Let's write the characteristic function:

$v(\{c\}) = v(\{m\}) = v(\{p\}) = 0$
$v(\{c, m\}) = v(\{c, p\}) = 750$
$v(\{m, p\}) = 500$
$v(\{c, m, p\}) = 1000$

We don't know though how these children will split the ice cream.
Let's introduce the concept of ***outcome***.
the outcome is a pair $(CS, \bar{x})$ where

- CS is the ***Coalition Structure***: simply a partition of the set of agents.
  $CS = \{c_1, c_2, c_3, \ldots, c_k\}$
  $U_{i=1}^{k} c_i = A \quad c_i \cap c_j = \{\} \; \forall 1 \leq i, j \leq j$
- $\bar{x}$ is the ***Payoff Vector***, $\bar{x} = (x_1, x_2, \ldots, x_n)$
  $\sum_{i \in C} x_i = v(C) \quad \forall C \in S$

Another example

$v(\{1, 2, 3\}) = 9$
$v(\{4, 5\}) = 4$
$((\{1, 2, 3\}, \{4, 5\}), (3, 3, 3, 3, 1))$ where the orange part is the CS and the red one is $\bar{x}$.
This outcome respects the efficiency principle which says that the sum of the individual payoffs should sum the coalition payoff.

Superadditivity

An important property that a TU game can have is superadditivity:

$$v(\{C \cup D\}) \geq v(C) + v(D)$$

It is always convenient, overall, to form the grand coalition.

I said overall because individual utilities could be lower than in other coalitions.
In this case $CS = S = \text{grand coalition}$.

is the game $v(C) = |C|^2$ superadditive? yes it is.

Convex Property

A game is convex when
$v(C \cup D) + v(C \cap D) \geq v(C) + v(D) \;\; \forall C, D \subseteq A$

If a game is convex then it is even superadditive because a superadditive game is a special case of convex game in which the intersection is always empty

Simple Property

$v(C) \in \{0, 1\} \; \forall C \in A$

How to choose the payoff vector?

Let's modify a little the introductory example:

There are 2 children.
Charlie has 4 dollars.
Marcy has 3 dollars.
Patty has 3 dollars.
Charlie wants to buy ice cream which is 500g and it costs 7 dollars. The others are explained mathematically.

$w_c = 500g;\ c_c = 7\$$
$w_m = 750g;\ c_m = 9\$$
$w_p = 1000g;\ c_p = 11\$$

$v(\{c\}) = v(\{m\}) = v(\{p\}) = 0$
$v(\{c, m\}) = v(\{c, p\}) = 500$
$v(\{m, p\}) = 0$
$v(\{c, m, p\}) = 750$

This game is superadditive so we can say that the grand coalition is going to form.

Now let's look how we can decide the payoff vector

$(A, (0, 0, 0))$ is possible? no, not efficient.
$(A, (200, 200, 350))$: the payoff vector is possible but it's not a good one because Charlie and Marcy are going to get less than Patty. Charlie and Marcy can split from Patty and get more each (250 instead of 200).
We say that the above outcome is not ***stable***.
A stable outcome is captured though the concept of ***core***.

Core

$$\text{core} = \{(CS, \bar{x} | \sum_{i \in C} x_i \geq v(C)\ \ \forall C \subseteq A\}$$

The core embeds the idea of stability.
So, given a certain $\bar{x}$ we check whether it is a solution acceptable by everyone.
the payoff $(250, 250, 250)$ is in the core because for example $250 \geq 0$ if we consider $C = \{1\}$.
Even the payoff $(750, 0, 0)$ is stable, but it is not **fair**!
Even $(250, 250, 250)$ is not completely fair because Charlie has more money than the others two so he should get more ice cream.

Example about stability

$A = \{1, 2, 3\}$
$v(C) = \begin{cases} 1\ \ if|C| \geq 2 \\ 0\ \ otherwise \end{cases}$

This game is superadditive and it has no solution in the core.
The core can tell you if a payoff vector is stable but it cannot tell you if it is not fair.
Property: *the core of a convex game is never empty*.

Shapley Value

the Shapley value embeds the idea of fairness

Example about fairness

$A = \{1, 2\}$
$v(\{1\}) = v(\{2\}) = 5$
$v(\{1, 2\}) = 20$
The game is superadditive.
A fair outcome is $((\{1, 2\}), (10, 10))$, but even $((\{1, 2\}), (5, 15))$ and many more.

Shapley Value

Let's introduce some terminology:

$\Pi^A = \text{permutations over } A$ (it is an ordering over the agents)

$c_\pi(a_i)$the number of the agents that in the ordering p are appearing before agent a_i

$\Delta_\pi(a_i) = v(C_\pi(a_i) \cup \{a_i\}) - v(c_\pi(a_i)) =$ marginal contribution of agent a_i

The formula is:

$$\varphi_i = \frac{1}{n!} \sum_{\pi \in \Pi^A} \Delta_\pi(a_i)$$

CCF Part 2 - 15/11

(A, v) with $A = \{a_1, \ldots, a_n\}$

$(CS, \bar{x})$

$\varphi_i = \frac{1}{n!} \sum_{\pi \in \Pi^A} \Delta_\pi(a_i)$

Shapley Value's Properties

- <u>Efficiency</u>
  $\varphi_1 + \varphi_2 + \cdots + \varphi_n = v(A)$
  a *Dummy Player* (null player) is an agent a_i such that:
  $v(C) = v(C \cup \{a_i\}) \quad \forall C \subseteq A \;\; a_i \notin C$
  If we have a dummy player then its Shapley Value is 0.
- <u>Symmetry</u>
  players a_i, a_j are symmetric if and only if
  $v(C \cup \{a_i\}) = v(C \cup \{a_j\}) \quad \forall C \subseteq A \quad a_i, a_j \notin C$
  In this case *their Shapley Values are equal*.
- <u>Additivity</u>

  Consider 3 different games
  $1 : (A, v_1)$
  $2 : (A, v_2)$
  $3 : (A, u = v_1 + v_2)$
  $\rightarrow \varphi_i^1 + \varphi_i^2 = \varphi_i^3$

Shapley Values are the only way to distribute the payoffs that respect all these 3 properties.

Can we have a situation in which we can not compute the Shapley Values?
No, we can always compute a payoff vector with such values.

Is this payoff vector always stable? no, but:

for convex games, the payoff vector composed of Shapley values is always stable.

In case of convex games we can say that fair outcomes are also stable in case of convex games.

Biggest issues

It's related to the representation of the characteristic function games (A, v).

for every subset of agents S, v should return a number. So if we have 100 agents we need 2^{100} numbers to be returned.

The problem is solve If the game is additive. It means that I have to compute v for each agent $\rightarrow$ consequently a coalition made of agents 1 and 2 will be the sum of the value associated to 1 and 2 separately.

Best Coalition Structure

Let's assume we have 3 agents: $A = \{a_1, a_2, a_3\}$. What are the possible coalitions?

$$\{a_1\}, \{a_2\}, \{a_3\}, \{a_1, a_2\}, \{a_1, a_3\}, \{a_2, a_3\}, \{a_1 a_2 a_3\}$$

How many possible coalition structures do we have? (a coalition structure is a partition of the set of agents)

$$\Big\{\{a_1\}\{a_2\}\{a_3\}\Big\}$$

$$\Big\{\{a_1, a_2\}\{a_3\}\Big\}$$

$$\Big\{\{a_1\}\{a_2, a_3\}\Big\}$$

$$\Big\{\{a_2\}\{a_1, a_3\}\Big\}$$

$$\Big\{\{a_1, a_2, a_3\}\Big\}$$

In general, what is the number of coalition structures? it is a number $n^{n/2} \leq \alpha \leq n^n$.
It is a very large space in which we are looking for.
Let's define the characteristic function for each coalition, because we'll need it for finding the best coalition structure.

$v(\{a_1\}) = 30$

$v(\{a_2\}) = 40$

$v(\{a_3\}) = 25$

$v(\{a_1, a_2\}) = 50$

$v(\{a_1, a_3\}) = 60$

$v(\{a_2, a_3\}) = 55$

$v(\{a_1, a_2, a_3\}) = 90$

This game is not superadditive so the grand coalition is not going to form $\rightarrow$ we need to find the best coalition structure.
The best coalition structure is

$$CS^* = \arg\max_{CS \in P^A} \sum_{C \in CS} v(C)$$

So, for this example we have:

$$\Big\{\{a_1\}\{a_2\}\{a_3\}\Big\} \rightarrow 90$$

$$\Big\{\{a_1, a_2\}\{a_3\}\Big\} \rightarrow 75$$

$$\Big\{\{a_1\}\{a_2, a_3\}\Big\} \rightarrow 85$$

$$\Big\{\{a_2\}\{a_1, a_3\}\Big\} \rightarrow 100$$

$$\Big\{\{a_1, a_2, a_3\}\Big\} \rightarrow 90$$

We select the last but one coalition!

The problem is that if the agents are a lot this is not easily computable because the dimension of P^A would be very huge.
How to find a more efficient solution?
There are two possible algorithms:

- Centralized Algorithm
- Distributed Algorithm

Centralized Algorithm based on DP

It is based on dynamic programming.

Notation:

$f(C) = \text{the value of the optimal partition of } C$, where C is a coalition.

$t(C) = \text{ the optimal partition of } C$

$$f(C) = \begin{cases} v(C) & \text{if } |C| = 1 \\ \max\left\{ v(C), \max_{\{C', C''\} \in P^C} \left\{ f(C') + f(C'') \right\} \right\} & \text{otherwise} \end{cases}$$

It is just enough if you iterate over all the possible partitions in two sets.

We'll exploit this result in the following algorithm:

We'll compute all f and t for all the coalitions of one agent. then we compute such values for all coalitions of two agents, and so on until we consider the coalition with n agents.

C	$f(C)$	$t(C)$
$\{a_1\}$	30	$\{a_1\}$
$\{a_2\}$	40	$\{a_2\}$
$\{a_3\}$	25	$\{a_3\}$
$\{a_1, a_2\}$	$^* = 70$	$\{a_1\}, \{a_2\}$
$\{a_1, a_3\}$	60	$\{a_1, a_3\}$
$\{a_2, a_3\}$	65	$\{a_2\}, \{a_3\}$
$\{a_1, a_2, a_3\}$	$** = 100$	$\{a_2\}, \{a_1, a_3\}$

I'm going to explain just one of the cases in which $|C| = 2$. in particular it is the one for which $C = \{a_1, a_2\}$

$^* =$ we are no more in the first case of f because the cardinality is 2, so let's do the other case:
We have just one possible pair (C', C''), so:

$$\max\left\{ v(\{a_1, a_2\}), \max\left\{ f(\{a_1\}) + f(\{a_2\}) \right\} \right\}$$

$$\max\left\{v(\{a_1,a_2\}),\left(f(\{a_1\})+f(\{a_2\})\right)\right\}$$

$$\max\left\{50,\left(30+40\right)\right\}=70$$

$**=$ Let's consider $C=A$:

We have 3 possible pairs (C',C''), so:

$$\max\left\{v(\{a_1,a_2,a_3\}),\max\left\{f(\{a_1\})+f(\{a_2,a_3\})\ ,\ f(\{a_2\})+f(\{a_1,a_3\})\ ,\ f(\{a_3\})+f(\{a_1,a_2\})\right\}\right\}$$

$$\max\left\{90,\max\left\{30+65\ ,\ 40+60\ ,\ 25+70\right\}\right\}$$

$$\max\left\{90,\max\left\{95\ ,\ 100\ ,\ 95\right\}\right\}$$

$$\max\left\{90,100\right\}=100$$

Drawbacks:

- The number of rows of the table are exponential.
- I've to look at all possible partitions in each subsets (I'm talking about the inner maximum operator) $\rightarrow$ this is not a problem most of the time because listing all possible partition in a subset with respect to the number of partitions of the entire set A.

Observations:

- sometimes v is not equal to f. this means that making a partition out of a coalition sometimes is convenient!

Distributed Algorithm (Shehory - Kraus)

$(1)\ C_i \leftarrow \text{coalitions that include } a_i$

$(2)\ C_i^* \leftarrow \arg\max_{C\in C_i} \frac{v(C)}{|C|}$

$(3)\ \text{broadcast}(a_i, C_i^*), \text{receive}(a_j, C_j^*)$

$(4)\ C_{max} \leftarrow \text{largest set of agents such that, for all } a_j \in C_{max}, (a_j, C_{max})$

$(5)\ \text{if } a_i \in C_{max} \text{ then join } C_{max} \text{ and return}$

$(6)\ \text{delete from } C_i \text{ coalitions that contain argents in } C_{max}$

$(7)\ \text{goto } (2)$

This algorithm is performed by all agents in parallel knowing the same information.

We'll talk about it next time.

Multi Agent Planning - 19/11

There is not a local objective for each agent but a global goal for the system.

We can say that agents are cooperative/benevolent.
We'll be building a plan: a sequence of actions.

You need to build a plan when your choice or the action that you perform now will affect the action you will perform far in the future.

2 big areas of planning

- Path planning
  you have a physical environment and you want to find a path in this environment.
  It can be geometrical or a graph.
  You have a sequence of actions/vertices that you have to visit one after an other.
- Task planning
  the actions are done to perform a task (example: moving a block from one place to another).

2 ways to represent a plan

- Explicitly
- Implicitly
  by using a policy.
  A plan is a policy in Markov decision process.

Single agent planning

you can have a planning model like STRIPS or like Markov decision problem.

Kind of knowledge you have to use in both:
you need to know a model of the environment and a model of how your actions change the environment.

For single agents this is difficult because the space in which you have to look for your plan grow exponentially with the number of steps of the plan $\rightarrow$ the problem of planning is hard for single agents.

Multi agent planning

When the action of an agent performed now affects the actions of the agents in the future.
What we need:

- Model of the environment
- Model of how your actions impact on the environment
- Some model of the other agent's plan (what the other agents are doing)

The space of possible plans grows in the number of possible actions and also in the number of agents.

Features of multi agent planning

1. Agents are cooperative.
   They try to maximize some joint performance measure, there is a global utility for the system that depends on the joint action performed by the agents.
2. There are some things that depend on single agents and other from a bunch of agents.

3. Locality.
   when you plan a sequence of actions for one agent, and for the second, and for the third, one action, performed by one agent, will affect the actions of a small amount of agents.

Multi Agent Planning Possibilities

Two main possibilities:

1. You have a multi agent aspect in producing the plan:
   every agent comes up with its own plan.
2. You have a multi agent aspect in executing the plan:
   I have a central entity (a supervisor) that builds the plan for the three agents.
   Then I will assign this plan to agents and they will execute the plan in a distributed way.

A combination of the two is also possible.

We will have a centralized planner and distributed execution

How can we build a plan for several agents?

$\downarrow$

Families of approaches to Multi Agent Planning

- Coordination prior to Local Planning
  I will set at the beginning some rules for coordination.
  Then I will leave the agents to decide their local plans.
  I'm assuming that the local plans are built by the agents.
  The coordination between local plans is achieved because everybody respects some (traffic) rules
  $\rightarrow$ "traffic rules" are "social laws" = rules that everybody knows.
  Somebody (the designer) has established social laws. Everybody in the system knows that and builds their plans according to social laws.
  If social laws are well designed, when agents build their local plan, this will be coordinated.
  You also have to assume that everybody else knows the traffic rules and assumes that everybody knows that you know the traffic rules $\rightarrow$ common knowledge.
  Organizational structure: this is what is done in companies: I impose some constraints on the freedom of the local plan. Rules are established first, if rules are good, everything is then coordinated.
- Local Planning prior to Coordination
  At the beginning agents make plans as they want, then they are merged and modified to make them coordinated.
  Different approaches to modify the plan: Freedom in planning $\rightarrow$ then modify.

In both cases is to try to detach the problem of planning and the problem of coordination.

- Decision Theoretic Multi Agent Planning
  We interleave at the same time local planning and coordination $\leftarrow$ *we will see this in details*
- Dynamic multi agent planning / Replanning / Interleaving Planning and Execution
  Each agent builds its own plan, every agent starts executing his plan.
  Agents decide what to do if something happens.

Stochasticity in Multi Agent Planning

Elements and observations:

- transition function:
  next state determined by current state + joint actions of all the agents

$\rightarrow$ I can provide a probability distribution over the next state
- The environment is not directly observable:
  Agents just receive some observations about the environment.
  They don't know in which state the environment is.
- There is a single reward function for the whole system which the agents try to maximize.

DecPOMDPs

DECentralized Partially Observable Markov Decision Problems.

- $\text{Set of Agents I } = \{1, 2, \dots, n\}$
- $\text{Set of States: } S = \{1, \dots\} \text{ not known by the agents directly}$
- $\text{Set of Actions agent } i \text{ can perform: } A_i$
  $\bar{A} = \times_{i=1,..,n} A_i$: vector of one action for each agents.
  Set of all possible Join Actions.
- $\text{Transition Function: } P : S \times \bar{A} \rightarrow \Delta S : \text{Set of all the probability distributions over the set } S$
  $P(s'|s, \bar{a})$ is the probability of reaching s' being in £s£ an performing the joint action $\bar{a}$
- Ω_i observations: Set of all possible perceptions agents i receives from environment.
  a Joint Observation is defined as $\bar{\Omega} = \times_{i=1,..,n} \Omega_i$
- $\text{Observation Function} \rightarrow O : S \times \bar{A} \rightarrow \Delta\bar{\Omega}$
  $O(\bar{o}|s, \bar{a})$ is the probability of receiving one specific observation for each agent $(\bar{o})$ when the world is in s and the joint action is $\bar{a}$.
- $\text{Reward Function} \rightarrow R(s, \bar{a})$
  What is the immediate reward the system gets when the world is in s and agents are jointly performing $\bar{a}$.
  R is global, it's the system's reward. Single agents do not have a reward.

This model is *Partially Observable* because the states are not known to the agents.

Example

The are 2 doors, behind one there is a tiger, behind the other a treasure.

If one agent opens the door with the tiger, both die $\rightarrow$ large penalty

If one agent opens the door with then money $\rightarrow$ they share the treasure

If both open the same door ....

$I = \{1, 2\} \text{ agents}$

$S = \{T_L, T_R\}$ (the tiger is behind the left door or behind the right door)

$A_i = \{O_L, O_R, L\}$

Open Left door, Open Right door, Listen.

$\bar{A} = \{< O_L, O_L >, < O_L, O_R >, \dots\}$

Transition function:

$P(T_L|T_L, < L, L >) = 1 \rightarrow$ probability the the tiger is behind left, given it was behind left and both agents are listening.

$P(T_R|T_L, < L, L >) = 0$

$P(T_L|T_L, < OR, L >) = 0.5 \rightarrow$ the game restarts randomly when a door is opened, like a next episode.

$P(T_R|T_L, < OR, L >) = 0.5$

$\Omega_i = \{hl, hr\}$ (the agent hears the tiger behind the left or the right door)

$O(< hl, hr > | < L, L >, T_L) =< 0.85, 0.15 > \rightarrow$ the tiger is behind the left door. both agents were listening to the left door. Agent 1 has the right perception with probability 0.85.

$R(< L, L >, *) = -2 \rightarrow$ both perform the listening action independently from where the tiger is. Listening has a cost.

$R(< L, O_R >, T_R) = -101$

continues in *Multi Agent Planning Part 2*.

Shapley Values Exercise Session - 22/11

In my OneNote notes.

Multi Agent Planning Part 2 - 26/11

Let's continue the example started in *Multi Agent Planning Part 1*.

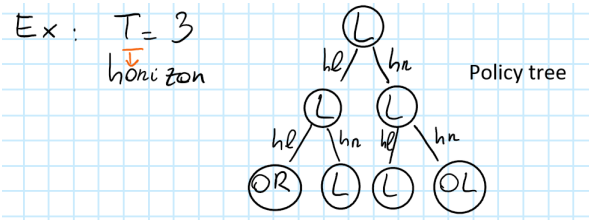
What is a solution to this problem?

In a MDP a policy is a function that given a state returns an action.

Right now, being the states not observable, the information that an agent has in this context is the sequence of observation he has done. So we have to change the definition of a policy for this kind of problems.

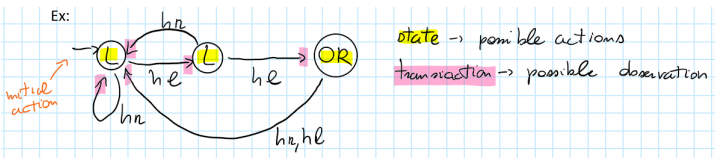
Policy
The policy q_i is a mapping from a sequence of observations to an action.

Finite Horizon
There's a finite number of actions in which the game can evolve. In this case a tree is the best possible representation for the policy



In this case an action OR or OL can be performed only after two listenings.
In this case we have assumed that the observations are the same for all the agents. The policy tree for the two agents will be binary (left or right for every node), but it's possible that the possible observations of different agents are different in general.

The drawn policy refers to agent 1. The policy of agent 2 is the same.



We can built different finite state controller with any number of states.
The one in the example is a finite state controller for a single agent.

How to find the optimal policy?

Given an initial state s_0 and a given $\bar{q}$, which is the joint policy (the policies set) I will define a function V that returns the expected reward.

$$V(s_0, \bar{q}) = E\left[\sum_{t=0}^{T-1} R(s_t, \bar{a}_t) | s_0, \bar{q}\right]$$

which can be read as the average over the sum of the rewards that I can get starting from a particular state and following a particular sequence of actions.
The objective is to find a joint policy $\bar{q}$ that maximizes the reward.

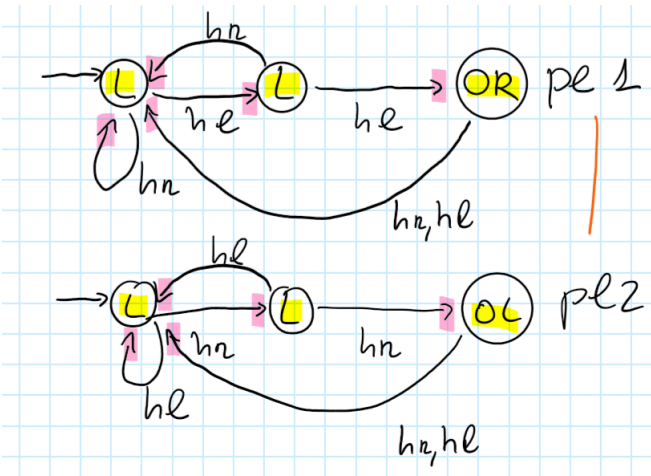
$$\bar{q}^* = \arg\max_{\bar{q}} V(s_0, \bar{q})$$

How difficult is this problem?
Given T (horizon $\rightarrow$ policy tree length) the complexity of solving this problem is $NEXP - hard$. This means super exponential (more than exponential). In fact I have to build every possible sequence of actions of length T for every agent evaluating the joint probability every time.

The goal becomes to find the best among all possible combinations of all the possible finite state controllers having T number of states:

$$V(s_0, \bar{q}) = E\left[\sum_{t=0}^{+\infty} d \cdot R(s_t, \bar{a}_t) | s_0, \bar{q}\right]$$

where $d = \text{discount factor}$.



In the case of 3 states those are the optimal policies for player 1 and player 2.
One agent checks one door, the other checks the other one and they join the knowledge coordinating themselves.
We will see a few steps of an actual algorithm to solve this kind of problems (this algorithm is not effective with high T).

$$V(s_0, \bar{q}) = R(s_0, \bar{a}) + \sum_{s', \bar{o}} P(s'|s, \bar{a}) O(\bar{o}|s, \bar{a}) V(s'|\bar{q}_{\bar{o}})$$

As we can see our expected reward ***depends*** on the transition function, and on the observation function.
joint policy composed of the sub-trees of the sub-trees of the initial policy.

The meaning of the term $P(s'|s, a)O(\bar{o}|s, \bar{a})$ is that we have to consider all the possible combinations of all the possible actions with all the possible observations.

The underlying idea is to try to solve the problems with an increasing dimension of the horizon:

$T = 1 \rightarrow$

Tree composed of a single level (the possible actions)$L\ (q_1), OR\ \ (q_2), OL(q_3) \rightarrow \bar{q} =< q_1, q_2, q_3$.
All possible combinations of joint policies $(\bar{q} \times \bar{q})$ are $9\ (3 \cdot 3)$.

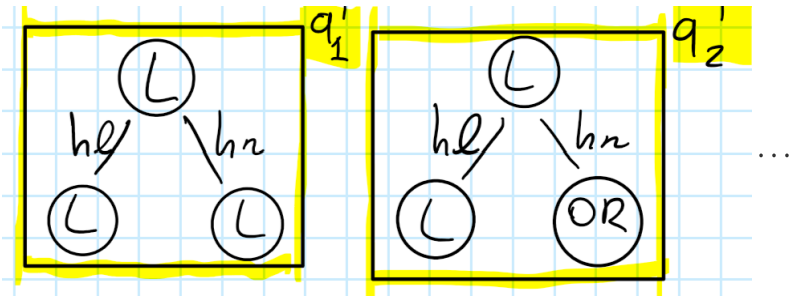
Fixed a particular joint policy $< q_1, q_1 >$
$V(TL, < q_1, q_1 >) = R(TL, < L, L >) = -2$.
In this case, on the right member of the equation, we do not have any term P, O, V because $T = 1$.

We have to compute V for all the 9 possible joint policies.

$V(TL, < q_1, q_2 >) = R(TL, < L, OR >) = 9$

and so on...

$T = 2 \rightarrow$

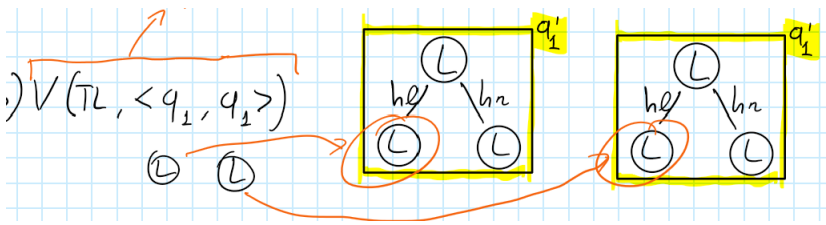


The total number of possible different trees for a single agent is 27.
The number of possible joint policies are $27 \cdot 27$.

Now consider we fixed the policy for agent 1: $< q_1', q_1' >$.

$V(TL, < q_1', q_1' >) =$
$R(TL, < L, L >) +$
$P(TL|TL, < L, L >)O(< hl, hl > |TL, < L, L >)V(TL, < q_1, q_1 >) +$
$P(TL|TL, < L, L >)O(< hl, hr >) \ldots +$
$P(TR|TL, \ldots) \ldots$

Keep in mind the meaning of the tuple $< q_1, q_1 >$:



The possible states are $TL, TR \rightarrow s'$.

The possible observations are $< hl, hl >, < hl, hr >, < hr, hr >, < hr, hl > \rightarrow \bar{o}$.

Considering even the one with probability associated $= 0$.

$$V(s_0, \bar{q}) = R(s_0, \bar{a}) + \sum_{s', \bar{o}} P(s'|s, \bar{a}) O(\bar{o}|, s, \bar{a}) V(s'|\bar{q}_{\bar{o}})$$

Then, all the q_i, q_j combinations have to be considered.

Then $T = 3$ has to be considered.

Possible ways of reducing complexity

- Consider the dominated and dominating policies. (Very complex, not described)

- Too complex to find the optimal policy $\rightarrow$ look for something simpler: *a joint equilibrium policy*.

Joint Equilibrium Policy

$< q_i, q_j >$ fixed for agent 1, find the optimal policy for agent 2.

$< q_i, q_j >$ fixed for agent 2, find the optimal policy for agent 1.

If I can find two policies such that given policy of player 1, player 2 can't find a better policy and vice versa that's an equilibrium.
The solution may not be optimal but the complexity is "just" exponential.